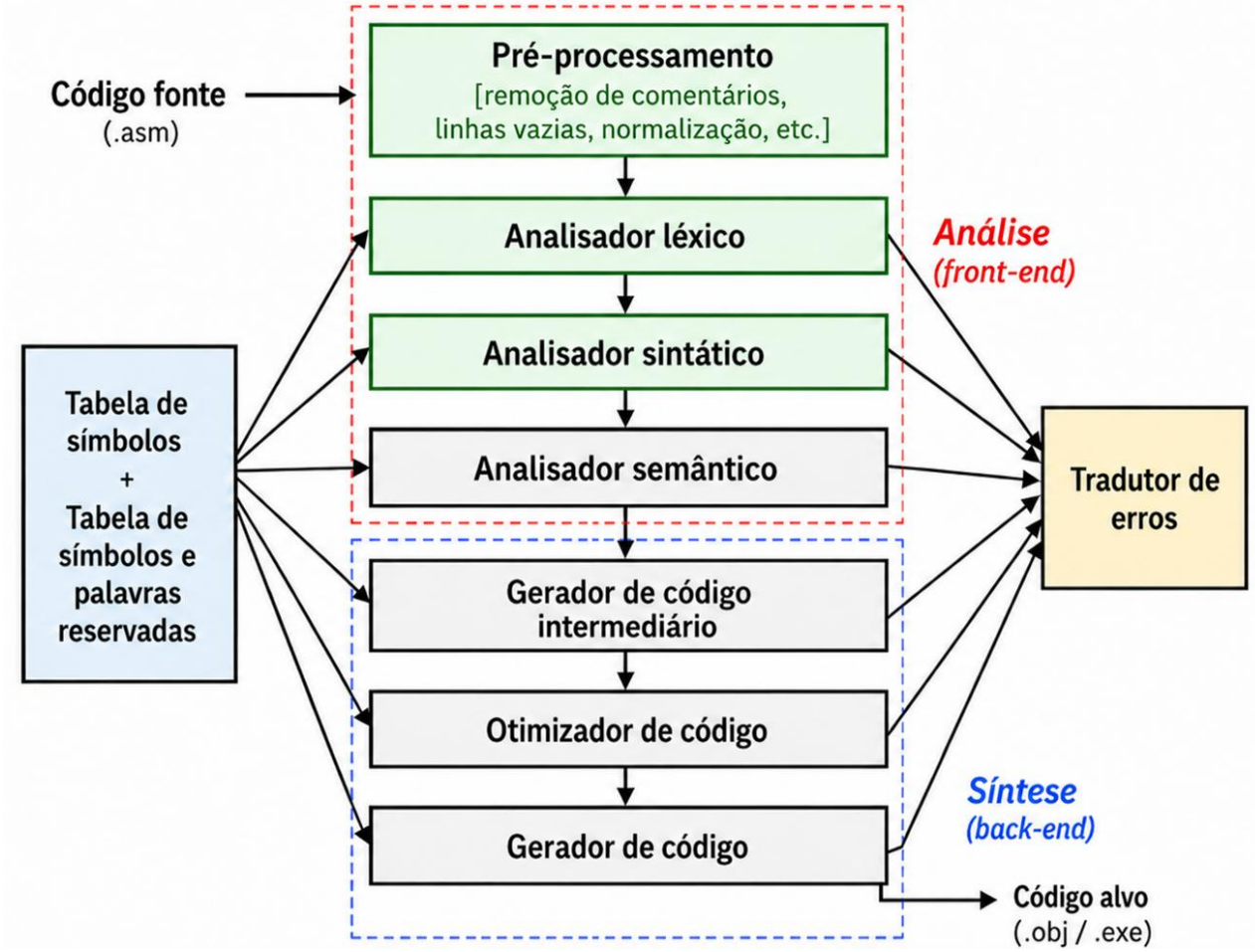
A ANÁLISE SINTÁTICA

1. Objetivo:

O objetivo deste trabalho é implementar, na linguagem C, um *analisador sintático para programas escritos em Assembly MIPS*, considerando o subconjunto de instruções e diretivas compatível com o simulador MARS especificado neste documento.



Localização da análise sintática dentro do processo de compilação

O analisador sintático deverá examinar a sequência de tokens produzida pelo analisador léxico e verificar se sua organização está de acordo com a gramática definida para a linguagem. Devem ser observadas as seguintes regras:

- O trabalho deverá ser desenvolvido pelo mesmo grupo formado para as fases prévias do compilador;
- Não será permitido utilizar ferramentas destinadas à geração automática de analisadores sintáticos;
- A implementação deverá ser integralmente realizada em C;
- Trabalhos total ou parcialmente iguais receberão avaliação nula;
- Ultrapassados cinco dias da data definida para a entrega, nenhum trabalho será recebido.

O analisador sintático deverá:

1	reconhecer as construções sintáticas do subconjunto da linguagem Assembly MIPS, conforme a gramática estabelecida;
2	verificar a organização das seções, das diretivas, das declarações de rótulos e das instruções;
3	verificar a quantidade, as categorias sintáticas e a ordem dos operandos de cada instrução;
4	identificar e registrar erros sintáticos, indicando linha, coluna, token encontrado e elemento esperado;
5	verificar a presença e a posição dos separadores e delimitadores exigidos pela gramática;
6	realizar a recuperação de erros, retomando a análise na próxima linha e permitindo a identificação de erros em construções posteriores;
7	consumir integralmente a sequência de tokens até TK_EOF, aceitando o programa somente quando não houver erros;
8	gerar, para programas sintaticamente válidos, um arquivo com extensão .dot, contendo a árvore sintática concreta representada na linguagem DOT do Graphviz;
9	gerar um arquivo .sin.err, contendo os erros sintáticos encontrados, ou uma mensagem indicando sua ausência; e
10	informar, ao término da análise, se o programa é sintaticamente válido ou inválido.

A implementação deverá utilizar a técnica de análise descendente recursiva, com funções correspondentes aos não terminais da gramática e decisões orientadas pelo próximo token da entrada, ou seja, o analisador sintático deverá implementar uma função para cada não terminal da gramática. Essas funções chamarão umas às outras conforme as regras de produção e consultarão o próximo token da entrada para decidir qual alternativa aplicar. Quando o token encontrado não corresponder ao esperado, o analisador deverá identificar um erro sintático.

A árvore sintática deverá apresentar o símbolo inicial da gramática como raiz, os não terminais como nós internos e os terminais como folhas. Cada folha deverá identificar a categoria do token e seu lexema, quando aplicável. Os filhos deverão respeitar a ordem dos símbolos na produção correspondente, e cada nó deverá possuir um identificador exclusivo.

Atenção:

Este trabalho está limitado à análise sintática. Portanto, não será responsabilidade do analisador verificar a existência de rótulos referenciados, a duplicidade de suas declarações, os limites numéricos dos operandos, a validade dos códigos de serviço utilizados em chamadas de sistema ou calcular endereços e gerar código de máquina. Tais verificações e operações pertencem às etapas de análise semântica ou de montagem.

2. Gramática

Uma gramática formal é um conjunto de regras que define como os símbolos de uma linguagem podem ser combinados para formar sentenças válidas. No contexto de uma linguagem de programação, essas regras descrevem a organização de instruções, declarações e outras construções.

Formalmente, uma gramática é representada pela quádrupla

$$\boldsymbol{G} = (\boldsymbol{N}, \boldsymbol{T}, \boldsymbol{P}, \boldsymbol{S}),$$

em que:

- **N** é o conjunto de símbolos não terminais, que representam construções da linguagem;
- **T** é o conjunto de símbolos terminais, que constituem as sentenças da linguagem;
- **P** é o conjunto de regras de produção, que estabelece como os símbolos podem ser substituídos;
- **S** é o símbolo inicial, a partir do qual são realizadas as derivações.

Por exemplo, uma instrução do Micro Assembler pode ser descrita pela produção:

<instrucao_li> ::= INS_LI REG SMB_COM NUM_INT

Nessa regra, <instrucao_li> é um não terminal, enquanto INS_LI, REG, SMB_COM e NUM_INT são terminais correspondentes às categorias de tokens produzidas pelo analisador léxico. A produção estabelece que a instrução deve conter, nessa ordem, a operação li, um registrador, uma vírgula e um número inteiro, como em li $t0, 10.

Importante

O analisador sintático utiliza a gramática para verificar se a sequência de tokens do programa apresenta uma estrutura válida. Nesta etapa do projeto de construção do compilador, deve ser desenvolvido um analisador sintático que consuma os tokens produzidos pelo léxico e verifique a organização das instruções, diretivas e rótulos conforme a gramática da linguagem.

Serão verificados:

- Reconhecimento das construções previstas na gramática.
- Verificação da quantidade, dos tipos sintáticos e da ordem dos operandos.
- Verificação de vírgulas, dois-pontos e parênteses.
- Mensagens de erro com localização, token encontrado e elemento esperado.
- Recuperação de erros para permitir a análise das construções seguintes.
- Casos de teste válidos e inválidos, acompanhados de relatório técnico.

A existência dos rótulos referenciados, sua duplicidade e os limites numéricos dos operandos não serão tratados nesta etapa.

2.1. A gramática a ser utilizada

A gramática apresentada neste documento define o subconjunto obrigatório da linguagem para esta etapa. Não será exigido o reconhecimento de construções adicionais aceitas pelo simulador MARS. O programa deverá apresentar pelo menos uma seção, iniciada por .data ou .text, admitindo-se seções vazias e alternância entre elas. Não será obrigatória a declaração de um rótulo denominado main.

As instruções somente serão admitidas na seção .text, e as diretivas de dados somente serão admitidas na seção .data. A diretiva .globl poderá ocorrer em ambas as seções, conforme a gramática. Cada linha lógica poderá conter, no máximo, uma instrução ou diretiva, precedida opcionalmente por um rótulo quando a produção correspondente permitir.

Os terminais apresentados entre aspas deverão ser reconhecidos pelas categorias correspondentes produzidas pelo analisador léxico. A categoria ID deverá ser utilizada nas posições destinadas a identificadores, inclusive na declaração e na referência a rótulos.

Convenções utilizadas

Notação	Significado
::=	Define uma produção.
|	Indica alternativas.
[...]	Indica uma parte opcional.
{ ... }	Indica zero ou mais repetições.
<nome>	Identifica um não terminal.
"texto"	Representa o token correspondente ao texto indicado.
ID	Identificador.
REG	Registrador.
NUM_INT	Número inteiro reconhecido pelo léxico.
STRING	Cadeia de caracteres reconhecida pelo léxico.
EOL	Término de uma linha lógica.
TK_EOF	Final da sequência de tokens.

Os terminais apresentados entre aspas deverão ser reconhecidos pelas categorias correspondentes produzidas pelo analisador léxico.

A categoria ID será utilizada nas posições destinadas a identificadores, inclusive na declaração e na referência a rótulos. Caso o léxico utilize uma categoria específica para rótulos, o módulo sintático deverá realizar a adaptação correspondente, preservando o lexema e sua localização.

2.2. Organização do programa

O programa deverá apresentar pelo menos uma seção, iniciada por .data ou .text e serão admitidas seções vazias e alternância entre elas. Ademais, as instruções somente serão admitidas na seção .text, e as diretivas de dados somente serão admitidas na seção .data. A diretiva .globl poderá ocorrer em ambas as seções.

Não será obrigatória a declaração de um rótulo denominado main.

```
<programa> ::= <secao> { <secao> } TK_EOF
<secao> ::= <secao_dados>
        | <secao_codigo>
<secao_dados> ::= ".data" EOL { <linha_dados> }
<secao_codigo> ::= ".text" EOL { <linha_codigo> }
<linha_dados> ::= <rotulo> [ <diretiva_dados> ] EOL
             | <diretiva_dados> EOL
             | <diretiva_global> EOL
<linha_codigo> ::= <rotulo> [ <instrucao> ] EOL
              | <instrucao> EOL
              | <diretiva_global> EOL
<rotulo> ::= ID ":"
<diretiva_global> ::= ".globl" ID
```

Cada linha lógica poderá conter, no máximo, uma instrução ou diretiva, precedida opcionalmente por um rótulo quando a produção correspondente permitir. Nesse caso, serão aceitos rótulos isolados e rótulos seguidos de uma construção na mesma linha:

```
.text
main:
   li $t0, 10
loop: addi $t0, $t0, -1
   bne $t0, $zero, loop
fim:
```

A descrição não torna obrigatório o nome main: ele continua sendo um identificador, e sua eventual exigência como ponto de entrada deve ser definida separadamente.

2.3. Declarações de dados

```
<diretiva_dados> ::= <diretiva_inteiros> <lista_inteiros>
                | <diretiva_string> STRING
                | ".space" NUM_INT
                | ".align" NUM_INT
<diretiva_inteiros> ::= ".word"
                  | ".half"
```

| ".byte"
<lista_inteiros> ::= NUM_INT { "," NUM_INT }
<diretiva_string> ::= ".ascii"
| ".asciiz"

Exemplos aceitos:

```
.data
numero:   .word 10
vetor:    .word 10, 20, -30, 0xFF
pequeno:  .byte 65
valor:    .half 1000
mensagem: .asciiz "Resultado: "
texto:    .ascii "ABC"
buffer:   .space 100
.align 2
```

Será aceita uma string por diretiva e uma lista de inteiros separados por vírgulas nas diretivas .word, .half e .byte. Não serão admitidas expressões, listas de strings ou repetição de valores por dois-pontos.

2.4. Instruções

As instruções podem ser agrupadas pela forma de seus operandos:

<instrucao> ::= <instrucao_tres_registradores>
| <instrucao_dois_registradores>
| <instrucao_um_registrador>
| <instrucao_imediato>
| <instrucao_deslocamento>
| <instrucao_li>
| <instrucao_la>
| <instrucao_memoria>
| <instrucao_desvio>
| <instrucao_salto>
| <instrucao_sem_operandos>

Operações com três registradores:

<instrucao_tres_registradores> ::= <op_tres_registradores> REG "," REG "," REG
<op_tres_registradores> ::= "add" | "addu" | "sub" | "subu" | "mul" | "and" | "or" | "xor" | "nor" | "slt"

Operações com dois registradores:

<instrucao_dois_registradores> ::= <op_dois_registradores> REG "," REG
<op_dois_registradores> ::= "move" | "div"

Neste trabalho, div utilizará exclusivamente a forma com dois operandos:

div $t0, $t1

Operações com um registrador:

<instrucao_um_registrador> ::= <op_um_registrador> REG
<op_um_registrador> ::= "mflo" | "mfhi" | "jr"

Operação com imediato:

<instrucao_imediato> ::= "addi" REG "," REG "," NUM_INT

Deslocamentos de bits:

<instrucao_deslocamento> ::= <op_deslocamento> REG "," REG "," NUM_INT
<op_deslocamento> ::= "sll" | "srl"

Carregamento de valor e endereço:

<instrucao_li> ::= "li" REG "," NUM_INT
<instrucao_la> ::= "la" REG "," ID

Acesso à memória:

<instrucao_memoria> ::= <op_memoria> REG "," <endereco>
<op_memoria> ::= "lw" | "sw" | "lb" | "sb" | "lh" | "sh"
<endereco> ::= ID | NUM_INT "(" REG ")"

Essa produção contempla as duas formas presentes nos materiais:

lw $t0, numero
lw $t0, 4($sp)

Desvios condicionais:

<instrucao_desvio> ::= <op_desvio> REG "," REG "," ID
<op_desvio> ::= "beq" | "bne" | "blt" | "ble" | "bgt" | "bge"

Saltos:

<instrucao_salto> ::= <op_salto> ID
<op_salto> ::= "j" | "jal"

Instruções sem operandos:

<instrucao_sem_operandos> ::= “syscall” | “nop”

3. Delimitação de linhas e seções

O analisador sintático deverá produzir internamente o marcador EOL, a partir da localização dos tokens fornecida pelo analisador léxico. Essa adaptação não deverá alterar o formato do arquivo .lex. Deverá ser inserido um marcador EOL após o último token de cada linha que contenha tokens, inclusive após a última linha do arquivo, mesmo quando não houver uma quebra de linha ao final.

Atenção:

Linhas vazias e linhas contendo somente comentários não deverão produzir marcadores adicionais.

As repetições de <linha_dados> e <linha_codigo> deverão terminar quando o próximo token representar a abertura de uma nova seção ou o final do arquivo. Uma construção não permitida na seção atual deverá ser diagnosticada como erro, e não interpretada como encerramento válido da seção.

O arquivo vazio ou composto exclusivamente por comentários deverá ser rejeitado, pois não apresenta a seção exigida pela produção <programa>.

Caso o arquivo .lex não contenha um token explícito de final de arquivo, o módulo sintático deverá acrescentar internamente TK_EOF. Caso esse token já esteja presente, não deverá ser duplicado.

4. Diagnósticos e recuperação de erros

Cada erro sintático deverá ser registrado com:

- Linha e coluna.
- Categoria e lexema do token encontrado.
- Token, categoria ou construção esperada.

Quando o erro corresponder a um elemento ausente ao final da linha, o diagnóstico deverá identificar essa linha e informar que foi encontrado seu término. Nesse caso, a coluna poderá ser a posição imediatamente posterior ao último lexema da linha. Para erros no final do arquivo, deverá ser informado que foi encontrado TK_EOF.

Por exemplo, para a linha abaixo, sem espaços iniciais:

add $t0 $t1, $t2

Um diagnóstico adequado será:

Linha 4, coluna 9:

Encontrado REG("$t1"); esperado SMB_COM(",").

Após detectar um erro, o analisador deverá descartar os tokens restantes da linha corrente até EOL e retomar a análise na linha seguinte, preservando a seção ativa. Quando for encontrada uma nova diretiva de seção, deverá reconhecer a mudança e prosseguir. Se o token corrente já for EOL no momento da identificação do erro, o analisador deverá consumir apenas esse marcador e retomar a análise na linha seguinte. Se for TK_EOF, deverá encerrar a recuperação. Em nenhuma dessas situações deverão ser descartados tokens de uma linha posterior.

A recuperação deverá garantir avanço na entrada, evitando repetição indefinida do mesmo diagnóstico. Não será exigida a identificação de todos os erros existentes em uma mesma linha; entretanto, erros em linhas posteriores deverão continuar sendo detectados.

Um programa que apresente qualquer erro sintático deverá ser considerado inválido, mesmo que a recuperação permita reconhecer as construções seguintes.

5. Limites entre sintaxe e semântica

O analisador deverá verificar a estrutura das construções, rejeitando, por exemplo:

```
add $t0, $t1        # falta um registrador
li $t0, mensagem    # exige um inteiro
lw $t0, 4 $sp       # faltam os parênteses
syscall $t0         # não admite operandos
.word 10,           # falta um inteiro após a vírgula
```

A última construção deverá ser testada dentro de uma seção .data, para que o erro avaliado seja a ausência do inteiro.

Não deverão ser tratados como erros sintáticos:

- Referências a rótulos inexistentes.
- Declarações duplicadas de rótulos.
- Inteiros fora dos limites de representação dos operandos.
- Quantidades de deslocamento inválidas em sll e srl.
- Valores semanticamente inválidos em .space e .align.
- Códigos de serviço inválidos ou não preparados antes de syscall.

Essas situações poderão ser sintaticamente válidas, desde que respeitem as produções apresentadas.

6. Árvore sintática concreta e arquivos de saída

Para cada programa sintaticamente válido, deverá ser produzido um arquivo .dot contendo sua árvore sintática concreta completa, representada na linguagem DOT do Graphviz.

A árvore deverá atender aos seguintes requisitos:

- A raiz deverá representar <programa>.
- Os nós internos deverão representar os não terminais.
- As folhas deverão representar os terminais reconhecidos, incluindo separadores, delimitadores, EOL e TK_EOF.
- Cada terminal deverá identificar sua categoria e seu lexema, quando aplicável.
- Cada ocorrência de um símbolo deverá possuir um identificador de nó exclusivo.
- Os filhos deverão respeitar a ordem dos símbolos na produção utilizada.
- As repetições da EBNF deverão aparecer como filhos sucessivos do nó correspondente.
- Partes opcionais ausentes não deverão gerar nós.
- Aspas, barras invertidas e demais caracteres especiais deverão ser escapados corretamente nos rótulos.

Para programas inválidos, a geração da árvore não será obrigatória, e o arquivo .dot deverá permanecer vazio. Não deverá ser apresentada uma árvore parcial como resultado de um programa válido.

O arquivo .sin.err deverá ser produzido em todas as análises sintáticas concluídas. Quando não houver erros, deverá conter:

Nenhum erro sintático encontrado.

Ao término da análise, o terminal deverá apresentar uma das mensagens:

PROGRAMA SINTATICAMENTE VÁLIDO

PROGRAMA SINTATICAMENTE INVÁLIDO

Serão avaliadas a validade do arquivo DOT, sua correspondência com a gramática, a ordem dos filhos e a presença dos terminais. A disposição visual dos nós não comporá a nota, pois depende do mecanismo de renderização do Graphviz.

Exemplo de programa e árvore

Considere:

```
.text
main:
   li $t0, 10
   li $t1, 20
   add $t2, $t0, $t1
   syscall
```

Sua árvore sintática concreta poderá ser representada pelo seguinte código:

```
digraph ArvoreSintatica {
    graph [ordering=out];
    node [shape=box, fontname="Arial"];
    edge [arrowhead=none];

    n0 [label="<programa>"];
    n1 [label="<secao>"];
    n2 [label="<secao_codigo>"];
    n3 [label="DIR_TEXT\n.text"];
    n4 [label="EOL"];
    n5 [label="<linha_codigo>"];
    n6 [label="<rotulo>"];
    n7 [label="ID\nmain"];
    n8 [label="SMB_COL\n:"];
    n9 [label="EOL"];
    n10 [label="<linha_codigo>"];
    n11 [label="<instrucao>"];
    n12 [label="<instrucao_li>"];
    n13 [label="INS_LI\nli"];
    n14 [label="REG\n$t0"];
    n15 [label="SMB_COM\n,"];
    n16 [label="NUM_INT\n10"];
    n17 [label="EOL"];
    n18 [label="<linha_codigo>"];
    n19 [label="<instrucao>"];
    n20 [label="<instrucao_li>"];
    n21 [label="INS_LI\nli"];
    n22 [label="REG\n$t1"];
    n23 [label="SMB_COM\n,"];
    n24 [label="NUM_INT\n20"];
    n25 [label="EOL"];
    n26 [label="<linha_codigo>"];
    n27 [label="<instrucao>"];
    n28 [label="<instrucao_tres_registradores>"];
    n29 [label="<op_tres_registradores>"];
    n30 [label="INS_ADD\nadd"];
    n31 [label="REG\n$t2"];
    n32 [label="SMB_COM\n,"];
    n33 [label="REG\n$t0"];
    n34 [label="SMB_COM\n,"];
    n35 [label="REG\n$t1"];
    n36 [label="EOL"];
    n37 [label="<linha_codigo>"];
    n38 [label="<instrucao>"];
    n39 [label="<instrucao_sem_operandos>"];
    n40 [label="INS_SYSCALL\nsyscall"];
    n41 [label="EOL"];
    n42 [label="TK_EOF"];
```

```
    n0 -> n1;
    n0 -> n42;
    n1 -> n2;
    n2 -> n3;
    n2 -> n4;
    n2 -> n5;
    n2 -> n10;
    n2 -> n18;
    n2 -> n26;
    n2 -> n37;
    n5 -> n6;
    n5 -> n9;
    n6 -> n7;
    n6 -> n8;
    n10 -> n11;
    n10 -> n17;
    n11 -> n12;
    n12 -> n13;
    n12 -> n14;
    n12 -> n15;
    n12 -> n16;
    n18 -> n19;
    n18 -> n25;
    n19 -> n20;
    n20 -> n21;
    n20 -> n22;
    n20 -> n23;
    n20 -> n24;
    n26 -> n27;
    n26 -> n36;
    n27 -> n28;
    n28 -> n29;
    n28 -> n31;
    n28 -> n32;
    n28 -> n33;
    n28 -> n34;
    n28 -> n35;
    n29 -> n30;
    n37 -> n38;
    n37 -> n41;
    n38 -> n39;
    n39 -> n40;
}
```

Os nomes das categorias dos terminais deverão corresponder aos utilizados pelo analisador léxico do projeto. O exemplo demonstra a estrutura sintática. Para encerrar a execução no MARS, seria necessário preparar o código de serviço em $v0 antes de syscall; essa condição não será verificada nesta etapa.

7. Entregáveis

O projeto deverá manter a separação entre os módulos de pré-processamento, análise léxica e análise sintática. A função main() será responsável por validar os argumentos, abrir e fechar os arquivos e coordenar a execução das etapas.

Deverá existir uma função de reconhecimento para cada não terminal da gramática. As repetições da EBNF poderão ser implementadas por estruturas de repetição, e as partes opcionais, por decisões orientadas pelo próximo token.

Código-fonte completo, contendo todos os arquivos .c e .h; arquivo principal; implementação do analisador léxico e sintático; conforme detalhado a seguir:

main.c	Com a função main(), responsável pelo recebimento e validação dos argumentos da linha de comando, pela abertura dos arquivos e pelo acionamento das funcionalidades do pré-processador, do analisador léxico e do analisador sintático;
preprocessador.c	Deverá conter a implementação das funções responsáveis pelas etapas de pré-processamento do código Assembly;
preprocessador.h	Deverá conter os protótipos das funções, constantes, tipos e demais declarações necessárias ao módulo de pré-processamento.
lexico.c	Deverá conter a implementação das funções responsáveis pela etapa de análise léxica do código Assembly;
lexico.h	Deverá conter os protótipos das funções, constantes, tipos e demais declarações necessárias ao módulo de análise léxica.
sintatico.c	Deverá conter a implementação das funções responsáveis pela etapa de análise sintática do código Assembly;
sintatico.h	Deverá conter os protótipos das funções, constantes, tipos e demais declarações necessárias ao módulo de análise sintática.

A análise sintática deverá utilizar a seguinte interface, declarada em sintatico.h e implementada em sintatico.c:

int AnaliseSintatica(FILE *tokens, FILE *arvore, FILE *erros);

Os parâmetros correspondem a:

- tokens: arquivo .lex, aberto para leitura.
- arvore: arquivo .dot, aberto para escrita.
- erros: arquivo .sin.err, aberto para escrita.

A função deverá retornar:

Retorno	Significado
0	Programa sintaticamente válido.
1	Programa sintaticamente inválido.
-1	Falha operacional que impediu a conclusão da análise.

O módulo poderá carregar os tokens em memória antes de iniciar o reconhecimento. Cada token deverá preservar sua categoria, seu lexema e sua localização e as funções dos módulos deverão utilizar os arquivos recebidos por parâmetro. A abertura e o fechamento dos arquivos serão responsabilidade de main().

As etapas anteriores deverão manter suas interfaces públicas. A integração deverá permitir que main() identifique a ocorrência de erros léxicos, por retorno, consulta ao estado do módulo ou outro mecanismo explicitamente documentado.

Caso sejam encontrados erros léxicos, a análise sintática não deverá ser executada. O programa deverá informar:

ANÁLISE SINTÁTICA NÃO EXECUTADA: EXISTEM ERROS LÉXICOS.

Atenção:

Além do código-fonte completo, o grupo deverá entregar um relatório técnico contendo a descrição das estruturas de dados, a correspondência entre os não terminais e as funções de reconhecimento, o tratamento de EOL e TK_EOF, a estratégia de diagnóstico e recuperação de erros e o processo de construção da árvore sintática concreta. Deverão ser apresentados os arquivos dos casos de teste, seus resultados esperados e obtidos, os diagnósticos gerados e os arquivos DOT dos programas válidos. Também deverá ser entregue pelo menos uma imagem de uma árvore sintática completa, renderizada pelo Graphviz, acompanhada das instruções de compilação e execução do projeto.

8. Casos de teste

Deverão ser apresentados, no mínimo três programas sintaticamente válidos e três programas lexicalmente válidos contendo erros sintáticos. Os programas poderão reunir vários casos, mas o conjunto de testes deverá apresentar a seguinte cobertura:

Categoria	Casos obrigatórios
Organização	Programa iniciado diretamente por .text; seções .data e .text; alternância e seção vazia.
Rótulos	Rótulo isolado; rótulo seguido de instrução ou diretiva de dados na mesma linha.
Diretivas	Todas as diretivas previstas na gramática.
Instruções	Todos os grupos de instruções e todas as operações previstas na gramática.
Operandos	Inteiros decimais e hexadecimais, strings, listas de inteiros e referências a identificadores.
Memória	Endereço por identificador e por deslocamento seguido de registrador entre parênteses.
Linhas	Linhas vazias, comentários e arquivo sem quebra de linha final.
Entrada inválida	Arquivo vazio e programa sem abertura de seção.
Seção incorreta	Instrução em .data e diretiva de dados em .text.
Estrutura incorreta	Operandos ausentes, excedentes, fora de ordem ou de categoria inadequada.
Pontuação	Vírgulas, dois-pontos ou parênteses ausentes; lista de inteiros terminada por vírgula.
Consumo completo	Tokens adicionais após uma construção completa.
Recuperação	Erros em linhas distintas, com reconhecimento de construções posteriores.
Limites da etapa	Referência a rótulo inexistente e declaração duplicada de rótulo, sem rejeição exclusivamente por esses motivos.

Todos os casos utilizados para avaliar erros sintáticos deverão ser lexicalmente válidos.

9. Compilação e execução

Considerando a organização apresentada, a compilação poderá ser realizada com:

gcc main.c preprocessador.c lexico.c sintatico.c -o main.exe

Os arquivos .h não precisam ser informados no comando, pois serão incluídos pelos arquivos .c correspondentes. O programa deverá receber dois argumentos:

./main.exe <arquivo_entrada.asm> <prefixo_saida>

Exemplo:

./main.exe teste.asm resultado

Nessa execução, deverão ser utilizados os seguintes arquivos:

Arquivo	Conteúdo
resultado.pre	Código resultante do pré-processamento.
resultado.lex	Tokens produzidos pelo analisador léxico.
resultado.ts	Tabela de símbolos da etapa léxica.
resultado.lex.err	Diagnósticos léxicos.
resultado.sin.err	Diagnósticos sintáticos.
resultado.dot	Árvore sintática concreta, quando o programa for válido.

A execução deverá seguir a ordem: (1) Pré-processamento; (2) Análise léxica e (3) Análise sintática, desde que não tenham sido encontrados erros léxicos.

Os arquivos utilizados como entrada de uma etapa deverão estar posicionados no início antes da leitura. Arquivos de saída de uma execução anterior deverão ser substituídos, evitando que resultados antigos sejam apresentados como atuais. Quando a análise sintática não for executada por causa de erros léxicos, o arquivo .dot deverá permanecer vazio, e o arquivo .sin.err deverá registrar o motivo da não execução.

A função **main()** deverá tratar argumentos incorretos e falhas de abertura, leitura, gravação ou alocação de memória, apresentando mensagens claras e encerrando a execução de forma controlada.

Os códigos de encerramento do programa deverão ser:

Código	Situação
0	Execução concluída e programa sintaticamente válido.
1	Rejeição por erros léxicos ou sintáticos.
2	Falha operacional ou uso incorreto da linha de comando.

Para renderizar uma árvore DOT, poderá ser utilizado:

dot -Tpng resultado.dot -o resultado.png

10. Grade de correção

A pontuação obtida em cada item será proporcional ao atendimento do respectivo requisito, sendo a nota final determinada pela soma dos percentuais alcançados nos diferentes quesitos avaliativos e posterior aplicação ao valor máximo estabelecido para esta etapa do projeto.

Nº	Quesito avaliativo	Pontuação máxima	Observações
1	Compilação, execução, organização modular e interface exigida	5%	
2	Integração com as etapas anteriores e leitura correta dos tokens	5%	
3	Análise descendente recursiva, com funções correspondentes aos não terminais	15%	
4	Reconhecimento de seções, rótulos e diretivas de dados	15%	
5	Reconhecimento das instruções, operandos, separadores e delimitadores	20%	
6	Tratamento de linhas, final do arquivo e consumo integral da entrada	5%	
7	Detecção e localização dos erros sintáticos	10%	
8	Recuperação de erros e continuidade da análise	10%	
9	Geração correta da árvore sintática concreta e das saídas obrigatórias	10%	
10	Relatório técnico e cobertura dos casos de teste	5%	
	Total	**100%**	

A aceitação de entradas inválidas e a rejeição de entradas válidas serão consideradas falhas de reconhecimento. Um analisador que apenas aceite ou rejeite todas as entradas não receberá pontuação por correção sintática.

A avaliação terá como foco o analisador sintático. As etapas anteriores serão verificadas quanto à integração necessária ao seu funcionamento.